

12 + 10

22

267549

Student ID: 202512100

Student Name: Priya-J. Shah

Dhirubhai Ambani Institute of Information & Communication Technology

Second In-Semester Examination, Autumn Semester 2025-2026

Course Title: IT637 Introduction to Algorithms

Max Marks: 55

Date: 20th Mar 2026

Time: 120 mins

Instructions:

- All questions are compulsory.
- Please write your answers in brief; long answers do not result in higher marks.
- All answers must be written in the space provided in the question paper only. Do not use supplements for the final answers.
- Do not write answers with a pencil. They will not be considered for grading.

Q1. [2 Marks Each]

- a. Given a social network directed graph, where vertices are people and edges connect a person with one or more persons who have similar interests, find a distinct group of people with similar interests. Which algorithm can be used to solve this problem? In a couple of sentences, explain your answer.

Answer: I think SCC (Strongly Connected Components) will be an appropriate choice for network graph connecting people with similar interests as it will cluster people with high similar interests. So, SCC will be giving subsets of people with high similar interests as they all will be interconnected to each other.

- b. Given a graph where the vertices are objects and the edges are the references to other objects, find the objects that are not referenced by any other objects. Which algorithm can be used to solve this problem? In a couple of sentences, explain your answer.

Answer: I think we can use BFS/DFS either one. As we will traverse through the graph all the connected vertices will be visited. Only those vertices (objects) that are not referenced will be unvisited which we can traverse through and end.

- c. If you needed to calculate the in-degree of a single vertex in a graph, which of the two representations (adjacency matrix or adjacency list) would you prefer? In a couple of sentences, describe why.

Answer:

I will use adjacency list. For a single vertex to store only the in-degree it will be a correct choice.

As it will use minimal space for a single vertex by making a list of only that vertex and easy to access & traverse. It won't be a good idea to use Adjacency matrix for a single vertex as it will consume more space & then ^{accessing} traverse also takes more time.

Student ID: 202512100
Student Name: Pritya J. Shah
d. If you need to represent a graph with a property of $E \ll V^2$, which of the two representations (adjacency matrix or adjacency list) would you prefer? In a couple of sentences, describe why.
matrix as it keeps track of the loop through

Student Name: Pritya J. S.

d. If you need to represent a graph with a property of Eulerian (adjacency matrix or adjacency list) would you prefer? In a couple of sentences, describe why.

Answer: we use adjacency matrix as it keeps track of the loop through diagonal of the matrix.

chance of cycle which can be used for this property.

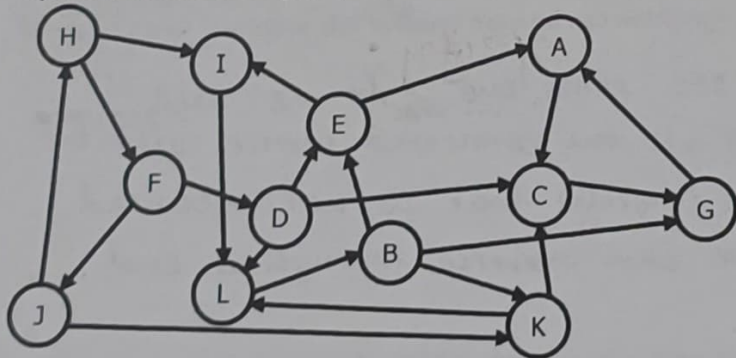
we use adjacency ~~list~~
diagonal of the matrix.
In list there is a chance of cycle which can be
~~dangerous~~, so matrix is a better choice for this property

e. DFS algorithm has time complexity of $O(|V| + |E|)$ and not $O(|V| * |E|)$. Explain with reason why $O(|V| + |E|)$ and not $O(|V| * |E|)$?
Ans: In DFS, we traverse all the nodes.

e. DFS algorithm has time complexity of $O(|V| + |E|)$ and not $O(|V| * |E|)$ and not $O(|V| * |E|)$?

Answer: In DFS, we go till depth to leaf node. We traverse all the edges in between only ~~twice~~ ~~once~~ while visiting them and another time while back-tracking to the parent. So it takes $O(|V| + |E|)$. We will not recursively go in and visit the already visited nodes. So it won't take $O(|V| * |E|)$ time. As it checks whether the node is visited or not before deep diving!

Q2. Use the following graph for each of the subproblems:



- a. Run the depth-first search (DFS) algorithm on the directed graph provided above, starting from vertex A first and then from vertex H to build a DFS forest. Assume that the adjacency list stores edges in alphabetical order, e.g., when exploring white out-neighbors of vertex D, the DFS algorithm considers the edge $D \rightarrow C$, then $D \rightarrow E$ and finally $D \rightarrow L$. Similarly, for vertex F, the order in which edges are considered for DFS is $F \rightarrow D$ and $F \rightarrow J$. During the construction of the DFS Forest keep track of discovery time and finish time of each vertices as they are pushed to the stack and popped out of the stack.

[6 Marks]

Answer: (On the next page)

Student ID: 202512100
 Student Name: Priya J. Shah

~~A-C-G~~

~~H-F-D-E-I~~

DFS:
~~A-C-G-H-F-D-E-I-L-B-K-J~~

0

	DT	FT
A	0	4
B	10	12
C	1	3
D	6	16
E	7	15
F	5	18
G	2	2
H	3	19
I	8	14
J	17	17
K	11	11
L	9	13

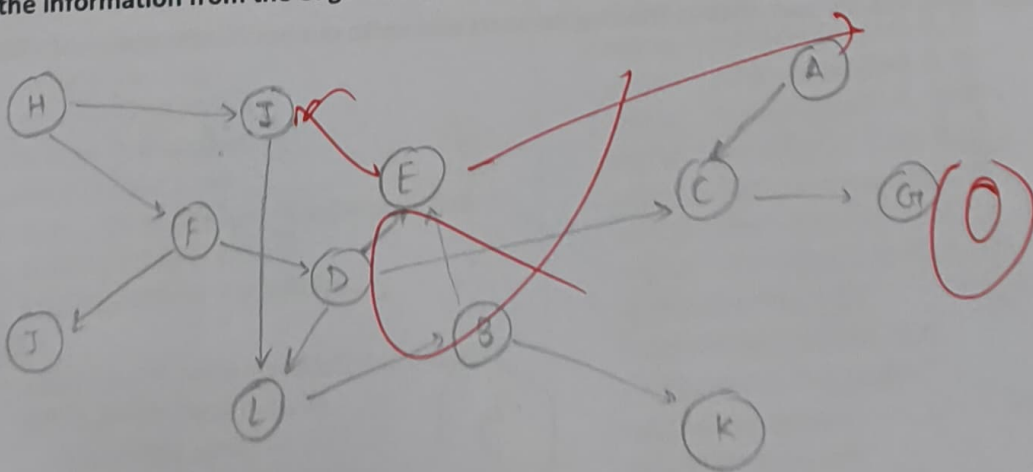
- b. Now using the discovery and finish times of the vertices recorded in (a), write the category of each edge as forward, backward, or cross edges in the table below. [5 Marks]

2

H → I	Forward	F → D	Forward	D → E	Forward	E → A	cross-edge
H → F	Forward	F → J	Forward	D → L	Forward	B → E	Backward
J → H	Backward	I → L	Forward	D → C	cross-edge	B → G	cross-edge
J → K	Backward	L → B	Forward	E → I	Forward	B → K	Forward
A → C	Forward	C → G	Forward	K → C	cross-edge	K → L	Backward
G → A	Backward						

- c. Draw a Directed Acyclic Graph (DAG) by converting the directed cyclic graph from (a) to an acyclic graph using the information from the edge classification from (b). [4 Marks]

Answer:



Student ID: 202512100

Student Name: Priya J. Shah

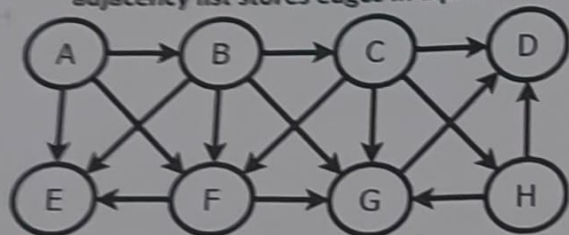
Q3.

- a. For the graph used in Q2, you have already executed DFS on the graph. Find the topological order starting from vertex A. Assume that the adjacency list stores edges in alphabetical order. If there are no valid orderings, state why not. [2 Marks]

Answer:

I think topological order no topological valid orderings are possible, because there are cycles detected in the graph which will not let us get to an answer.

- b. Run DFS on the graph below to find the topological order starting from vertex A. Assume that the adjacency list stores edges in alphabetical order. If there are no valid orderings, state why not. [5 Marks]



Answer:

A → B, E, F
B → C, F, G
C → D, G, H
D →
E →
F → G
G → D
H → D, G

A	A
B	B
C, G, H	C
A, B, F	D
A, B, C	E
B, C, F, H	F
	G
	H

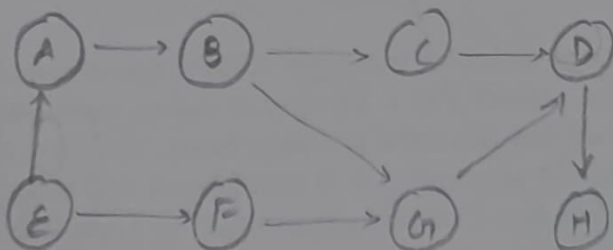
A → H → B → C → F → G → D → E

A → B → C → F → E → H → D → G

0

- c. Draw a graph with exactly two distinct topological sorts and write out both orderings. [3 Marks]

Answer:



E - A - B - C - F - G - D - H
E - F - A - B - C - G - D - H

3

Student ID: 202512100

Student Name: Priya. J. Shah

Q4.

Consider the partial code provided below that represents directed graph G using the adjacency list implementation. Partial codes for the methods are already provided in the table below: (5 Marks Each)

There are two tasks in this question:

1. Given a directed graph $G = (V, E)$ with number of vertices V and number of edges E , generate adjacency list implementation of the reverse graph GR in `adjListTransposed` by reversing all edges of the original graph given by adjacency list `adjList`, known as the transpose of a graph. You do not need to modify the original graph, but just create a new adjacency list for the reverse graph. Implement the code in the empty method `transpose()` provided below.
2. Empty method `RemoveEdge()` is provided to you. Write a code to remove the edge between the source and destination vertices from the adjacency list. Display an appropriate error message if the edge between the given vertices is not found. (Hint: You need to consider 3 scenarios: (a) edge not found, (b) edge found for the first node of the list, (c) edge found but not at the first node.)

C1	// Definition of AdjListItem: stores the // head node of the linked list for each // vertex (MAX_VERTICES) AdjListItem.src_vertex AdjListNode first;	C2	// Definition of AdjListNode: linked list node // that contains edge information AdjListNode.dst_vertex AdjListNode.next
C3	// Global Variables MAX_VERTICES = 5 AdjListItem[MAX_VERTICES] adjList AdjListItem[MAX_VERTICES] adjListTransposed	C4	// AddVertices(adjList) that adds all vertices // to adjList AddVertices(adjList) FOR i = 0 to MAX_VERTICES-1 adjList[i].src_vertex = i adjList[i].first = null RETURN adjList
C5	// AddEdge adds a new edge between // src and dst vertices by updating // adjList and returns the updated list AddEdge(adjList, src, dst) // create new edge AdjListNode newEdge newEdge.dst_vertex = dst newEdge.next = null // add edge to the end of adjacency list IF adjList[src].first == null adjList[src].first = newEdge ELSE AdjListNode node = adjList[src].first While (node.next != null) node = node.next node.next = newEdge	C6	// Main Code adjList = AddVertices(adjList); // Add edges adjList = AddEdge(adjList, 0, 1) adjList = AddEdge(adjList, 0, 4) adjList = AddEdge(adjList, 0, 3) adjList = AddEdge(adjList, 2, 0) adjList = AddEdge(adjList, 3, 2) adjList = AddEdge(adjList, 4, 1) adjList = AddEdge(adjList, 4, 3) // Remove the edge RemoveEdge(adjList, 0, 2) // Transpose the graph adjListTransposed = Transpose adjListTransposed)

Student ID: 202512100

Student Name: Priya J. Shah

RETURN adjList

// Fill in your code here for transpose i.e. reverse of the graph
Transpose(adjList, adjListTransposed)

adjListTransposed = AddVertices(adjListTransposed)

~~adjListTransposed~~

for i=0 to MAX-VERTICES-1

AdjListNode temp = adjList[i].first

adjListTransposed = AddEdge(adjListTransposed, temp.dst-vertex,
 adjList[i].src-vertex)

temp = temp.next

return adjListTransposed

// Fill in your code here for RemoveEdge
RemoveEdge(adjList, src, dst)

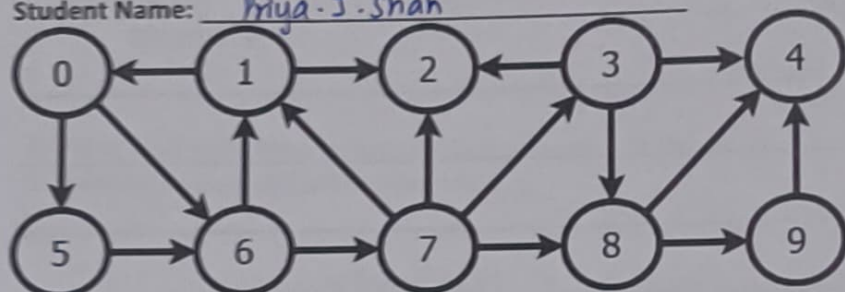
Return adjList

Q5.

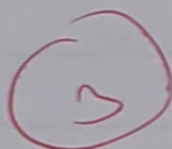
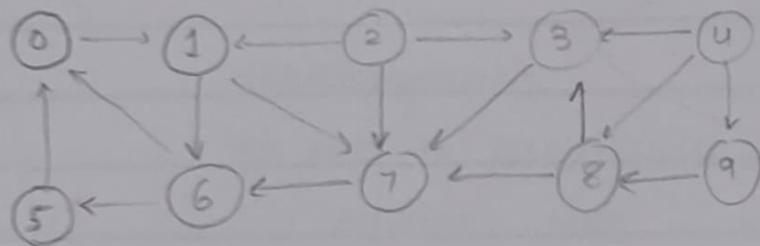
- a. On the graph given below, execute the Transposed algorithm you have written in Q4 and draw the transposed graph G^R . [3 Marks]

Student ID: 0902512100

Student Name: Priya J. Shah



Answer:



b. Create a list of vertices in topological order by running topological order on G^R .

[3 Marks]

Answer:

I think topological sort is not possible here because there is a cycle in the graph which do have vertices being interdependent on each other.

Like (0) needs (5,6)

(5) needs (6)

(6) needs (1,7)

(1) needs (0,2)

so somehow a cycle is being detected which won't allow topological sort.

c. List all the strongly connected components (SCCs).

[4 Marks]

Answer:

{0, 1, 5, 6, 7}

{2}

{3}

{4}

{8}

{9}

